C E R E M E M O R Y

A R C H I T E C T U R E D E C I S I O N R E C O R D S

Technology Stack and Core Engineering Decisions

Version 1.0 | March 2026

Author: Masato Okuwaki | CORe Inc.

Companion to: Cerememory Whitepaper v0.1 and CMP Spec v1.0

ADR-001: Core Implementation Language

Status	**Accepted**
Date	2026-03-12
Decision	**Rust as the primary implementation language for Cerememory core**

Context

Cerememory requires a language for its core engine that satisfies an unusually demanding set of constraints simultaneously. The engine must perform real-time decay computation across potentially millions of memory records. It must execute parallel queries across multiple heterogeneous storage engines. It must guarantee memory safety, because corruption of the memory database is catastrophic and unrecoverable. It must be embeddable as a library (local-first architecture). It must compile to native binaries for macOS, Linux, and Windows without runtime dependencies. And it must be efficient enough to run on consumer hardware without degrading the host system.

Options Considered

Option A: Rust

- **Zero-cost abstractions and no garbage collector:** predictable latency with no GC pauses, critical for real-time decay tick processing.
- **Ownership system:** compile-time memory safety without runtime overhead. Eliminates entire classes of data corruption bugs.
- **Fearless concurrency:** the borrow checker prevents data races at compile time. Essential for parallel multi-store queries.
- **Excellent cross-compilation:** native binaries for all target platforms with no runtime dependencies.
- **Strong ecosystem for embedded databases:** sled, redb, tantivy, and others are production-quality Rust-native storage engines.
- **FFI capabilities:** C-compatible ABI enables bindings from Python, Node.js, Go, and other languages for the adapter layer.
- **Concern:** steeper learning curve may slow early community contributions.

Option B: Go

- **Simpler concurrency model (goroutines):** easier to write parallel code, lower barrier for contributors.
- **Fast compilation:** faster development iteration cycles.

- **Concern:** garbage collector introduces unpredictable latency spikes. For a system processing continuous decay ticks across millions of records, GC pauses are unacceptable for the latency profile we need.
- **Concern:** less expressive type system. Cerememory's multi-store architecture benefits heavily from algebraic data types and trait-based polymorphism.
- **Concern:** larger binary size and runtime dependency (Go runtime).

Option C: C++

- **Maximum performance potential:** zero-overhead abstractions, mature optimizer toolchain.
- **Massive ecosystem:** decades of libraries for storage, indexing, and numerical computing.
- **Concern:** no compile-time memory safety. Manual memory management in a system where data corruption is catastrophic is an unacceptable risk. Memory safety bugs in database engines have historically caused data loss.
- **Concern:** undefined behavior is silent and difficult to detect. The complexity of Cerememory's multi-store concurrent access patterns would make C++ UB a persistent threat.

Option D: Python (with C extensions for hot paths)

- **Largest AI ecosystem:** seamless integration with PyTorch, transformers, vector databases, and the broader ML stack.
- **Lowest barrier to contribution:** the majority of AI developers write Python.
- **Concern:** orders of magnitude too slow for the core engine. The decay engine alone requires per-record floating-point operations on every tick. Python's GIL prevents true parallelism. Even with C extensions, the architecture becomes a fragile Python-C hybrid.
- **Concern:** not embeddable as a native library. Requires Python runtime on the user's machine.

Option E: TypeScript/Bun

- **Excellent for LLM integration:** most LLM SDKs have first-class TypeScript support.
- **Concern:** single-threaded event loop cannot handle sustained parallel computation across stores. Not suitable for the core engine, though appropriate for the adapter layer.

Decision

Rust is selected as the core implementation language. The decision is driven primarily by the intersection of three non-negotiable requirements: memory safety (data integrity), zero-cost concurrency (parallel multi-store queries), and embeddability (local-first, no runtime dependencies). Rust is the only language that satisfies all three simultaneously.

The adapter layer and LLM integration layer will provide bindings for Python, TypeScript, and other languages via Rust's C-compatible FFI. This means the AI ecosystem's language preferences are accommodated at the interface level while the core engine benefits from Rust's guarantees.

Consequences

- **Positive:** compile-time safety guarantees eliminate entire classes of runtime bugs. No GC pauses during decay processing. Single static binary deployment.
- **Positive:** Rust's async ecosystem (tokio) provides excellent support for the concurrent multi-store architecture.
- **Negative:** higher barrier to entry for contributors. Mitigated by providing Python/TypeScript SDKs so that adapter and integration work does not require Rust expertise.
- **Negative:** longer initial development time compared to Go or Python. Accepted as a tradeoff for long-term correctness and performance.

ADR-002: Embedded Multi-Engine Storage Architecture

Status	**Accepted**
Date	2026-03-12
Decision	**Embed specialized storage engines per store type rather than depend on external databases**

Context

Cerememory's architecture defines five distinct stores (episodic, semantic, procedural, emotional, working), each with different data models, query patterns, and performance characteristics. The system must be local-first with zero external dependencies. Users should not need to install PostgreSQL, Redis, Neo4j, or any other database server to run Cerememory.

Options Considered

Option A: External Database Services

Use PostgreSQL for episodic, Neo4j for semantic, Redis for working memory, etc. Each store backed by its optimal external database.

- **Concern:** violates local-first principle. Requires users to install and manage multiple database servers. Deployment complexity becomes prohibitive.

Option B: Single Embedded Database

Use a single embedded database (e.g., SQLite, RocksDB) for all stores, with different table schemas per store type.

- **Concern:** forces incompatible data models into a single engine. Graph queries on a key-value store are inefficient. Time-series queries on a relational engine require extensive workarounds. The "one engine for all" approach contradicts the neuroscience-inspired multi-store philosophy.

Option C: Embedded Multi-Engine (Selected)

Embed a specialized storage engine for each store type, all compiled into a single binary. Each engine is optimized for its store's data model and query patterns.

Decision

Cerememory will embed multiple specialized storage engines within a single binary. The following mapping is adopted as the starting point for the reference implementation:

Store	Engine	Rationale
Episodic	redb + custom time index	redb provides ACID transactions with zero-copy reads. Custom B-tree index on timestamps enables efficient temporal range queries. Fidelity scores stored inline for decay engine efficiency.
Semantic	Custom graph engine on redb	Adjacency list representation with typed edges stored in redb tables. Spreading activation implemented as breadth-first traversal with weight-based pruning. No existing embedded graph DB in Rust meets our activation propagation requirements.
Procedural	redb (profile tables)	Statistical profiles are relatively simple key-value structures with distributional data. redb's sorted tables support efficient range scans for pattern matching.
Emotional	Cross-cutting index on redb	Emotion vectors are metadata attached to records in other stores. Stored as a secondary index mapping record IDs to emotion vectors, with dimensional indices for emotion-filtered queries.
Working	In-memory (no persistence)	Working memory is volatile by design. A bounded concurrent HashMap (dashmap or similar) provides the fastest possible access with automatic eviction. No disk I/O.
Embeddings	usearch or custom HNSW	Vector similarity search for content-based recall. usearch provides a lightweight, embeddable HNSW index. If additional control is needed (e.g., fidelity-weighted search), a custom implementation on top of the base index.
Full-Text	tantivy	Tantivy is a Rust-native full-text search engine (Lucene equivalent). Provides tokenization, BM25 ranking, and faceted search for text-based recall cues.

Consequences

- **Positive:** zero external dependencies. Single binary contains everything. Users install Cerememory and it works.
- **Positive:** each store uses the optimal data structure for its access patterns. No impedance mismatch.
- **Positive:** all stores share the same process memory space, enabling zero-copy cross-store references and efficient association traversal.
- **Negative:** significant engineering effort to build and maintain multiple storage engines. Mitigated by leveraging existing Rust crates (redb, tantivy, usearch) rather than building from scratch.
- **Negative:** binary size will be larger than a single-engine approach. Acceptable for desktop/server deployment; may need optimization for future mobile targets.

ADR-003: Async Runtime and Concurrency Model

Status	**Accepted**
Date	2026-03-12
Decision	**Tokio as the async runtime with a dedicated thread pool for the decay engine**

Context

Cerememory has two distinct concurrency requirements that must coexist:

- **I/O-bound concurrency:** CMP operations (encode, recall, introspect) are request-response patterns that involve disk I/O across multiple stores. These benefit from async I/O with many concurrent tasks.
- **CPU-bound computation:** the decay engine performs continuous floating-point operations across all records. Spreading activation requires graph traversal with weight calculations. These are compute-intensive and must not block the I/O path.

Decision

The reference implementation will use Tokio as its async runtime for all CMP operation handling. The decay engine and consolidation engine will run on a dedicated thread pool (rayon or a custom pool) separate from the Tokio executor, communicating with the main system via channels.

This dual-pool architecture ensures that heavy decay computation never starves incoming CMP requests, and that I/O operations never delay decay ticks.

```
// Architectural sketch

// Pool 1: Tokio async runtime
//   - CMP request handling (encode, recall, lifecycle, introspect)
//   - Network I/O (HTTP/gRPC transport bindings)
//   - Disk I/O (store reads/writes via async wrappers)

// Pool 2: Dedicated compute threads (rayon)
//   - Decay engine ticks
//   - Consolidation cycles
//   - Spreading activation computation
//   - Fidelity recalculation batches

// Communication: tokio::sync::mpsc channels
//   - CMP ops send activation events to Pool 2
//   - Pool 2 sends fidelity updates back to Pool 1
```

Consequences

- **Positive:** clean separation of I/O and compute workloads. Neither can starve the other.
- **Positive:** Tokio is the de facto standard Rust async runtime with excellent ecosystem support (tonic for gRPC, axum/hyper for HTTP).
- **Positive:** rayon provides work-stealing parallelism ideal for the batch-processing nature of decay ticks.
- **Negative:** two-pool architecture adds complexity to the codebase. Requires careful channel design to avoid backpressure issues.

ADR-004: Spreading Activation Algorithm

Status	**Accepted**
Date	2026-03-12
Decision	**Weighted breadth-first spreading activation with configurable decay factor and hop limit**

Context

The cross-modal association engine is central to Cerememory's recall mechanism. When a memory is activated, related memories must be found by propagating activation through the association graph. This must be fast (sub-100ms for typical queries), bounded (cannot traverse the entire graph), and weighted (stronger associations propagate more activation).

Decision

Spreading activation is implemented as a weighted breadth-first search (BFS) with the following mechanics:

- **Initial activation:** the recalled record receives activation = 1.0.
- **Propagation:** at each hop, activation is multiplied by the association weight and a global decay factor (default: 0.5). Activation at hop N = parent_activation * edge_weight * decay_factor.
- **Threshold:** records with activation below a configurable threshold (default: 0.05) are pruned from the search.
- **Hop limit:** maximum traversal depth is configurable via the CMP activation_depth parameter (default: 2).
- **Fidelity boost:** activated records receive a temporary fidelity boost proportional to their activation level. This is the mechanism by which decayed memories become retrievable when related memories fire.
- **Cross-store traversal:** associations can link records in different stores. The activation engine dispatches parallel retrievals across stores when cross-store associations are encountered.

Consequences

- **Positive:** bounded computation with predictable latency. Worst case is O(branching_factor^depth) which is manageable with depth 2-3.
- **Positive:** naturally models the brain's spreading activation where distant associations receive weaker signals.

- **Negative:** BFS explores broadly rather than deeply. Some deep causal chains may not be reached within the default hop limit. Mitigated by allowing callers to increase activation_depth for specific queries.

ADR-005: Memory Decay Mathematical Model

Status	**Accepted**
Date	2026-03-12
Decision	**Modified Ebbinghaus power-law decay with emotional modulation and spaced repetition reinforcement**

Context

The decay engine is what makes Cerememory's memories "alive." The mathematical model must capture three phenomena: time-based forgetting, emotional modulation of forgetting rate, and reinforcement through repeated access (spaced repetition effect).

Decision

The fidelity score F of a memory record at time t is computed as:

```
F(t) = F_0 * (1 + t/S)^(-d) * E_mod

Where:
  F_0   = initial fidelity at encoding (default: 1.0)
  t     = time elapsed since last access (seconds)
  S     = stability constant (increases with each retrieval)
  d     = decay exponent (default: 0.3, configurable per store)
  E_mod = emotional modulation factor (1.0 + emotion.intensity * 0.5)
```

The stability constant S increases with each retrieval, implementing the spaced repetition effect:

```
S_new = S_old * (1 + retrieval_boost * S_old^(-0.2))

Where:
  retrieval_boost = configurable constant (default: 1.5)
```

This means frequently accessed memories become progressively more stable, requiring increasingly long intervals before significant decay occurs. This mirrors the well-established spacing effect in human memory research.

Noise accumulation is modeled as:

```
N(t) = N_0 + interference_rate * sqrt(t) * (1 - F(t))

Where:
```

```
N_0              = initial noise level (default: 0.0)
interference_rate = based on number of semantically similar records
sqrt(t)          = noise grows sublinearly (faster initially, then slower)
(1 - F(t))       = noise has more impact as fidelity drops
```

Consequences

- **Positive:** power-law decay is a better empirical fit to human forgetting data than simple exponential decay, especially for long time intervals.
- **Positive:** emotional modulation is a simple multiplicative factor, easy to compute and adjust.
- **Positive:** the stability-increasing mechanism naturally produces the spaced repetition curve without requiring an explicit scheduling algorithm.
- **Negative:** the model has several tunable parameters that will require empirical calibration. Initial defaults are based on published cognitive psychology literature but may need adjustment based on real-world usage patterns.

ADR-006: LLM Adapter Architecture

Status	**Accepted**
Date	2026-03-12
Decision	**Trait-based adapter pattern with SDK bindings in Python and TypeScript**

Context

The LLM adapter layer must translate between Cerememory's internal memory representation and the input/output formats of various LLMs. Each LLM has different context window sizes, prompt formats, system message conventions, and embedding spaces. The adapter must be model-specific but implement a standardized interface.

Decision

The adapter layer is defined as a Rust trait (interface) that model-specific implementations must satisfy:

```
pub trait LLMAdapter: Send + Sync {
    /// Serialize memories into LLM-consumable format
    fn serialize_context(
        &self,
        memories: &[RecalledMemory],
        budget_tokens: usize,
    ) -> AdapterOutput;

    /// Extract memory-worthy content from LLM output
    fn parse_output(
        &self,
        output: &LLMOutput,
        session_context: &SessionContext,
    ) -> Vec<EncodeStoreRequest>;

    /// Estimate token count for memory content
    fn estimate_tokens(&self, content: &MemoryContent) -> usize;

    /// Model metadata
    fn model_info(&self) -> ModelInfo;
}
```

Reference adapter implementations will be provided for Claude (Anthropic), GPT-4 (OpenAI), and Gemini (Google). Community contributions of additional adapters are encouraged.

For broader accessibility, SDK bindings will be published:

- **Python SDK (PyO3):** enables Python developers to use Cerememory and write custom adapters without Rust. Targets the AI/ML community.
- **TypeScript SDK (napi-rs):** enables Node.js/Bun developers to integrate Cerememory into web-adjacent applications and LLM toolchains.
- **C FFI:** raw C-compatible function interface for integration with any language that supports C FFI (Go, Swift, Java/JNI, etc.).

Consequences

- **Positive:** core engine stays in Rust with full safety guarantees. Adapter and integration code can be written in the contributor's preferred language.
- **Positive:** the trait interface is minimal and focused. Implementing a new adapter requires implementing only three methods.
- **Negative:** maintaining SDK bindings for multiple languages is an ongoing effort. Mitigated by auto-generating bindings where possible and prioritizing Python and TypeScript as the two highest-impact targets.

ADR-007: Serialization and Archive Format

Status	**Accepted**
Date	2026-03-12
Decision	**MessagePack for internal serialization; CMA (Cerememory Archive) format for export/import based on SQLite container**

Context

Two distinct serialization needs exist: internal serialization (how records are stored on disk within the engine) and external serialization (the portable archive format for export/import/backup).

Decision: Internal Serialization

MessagePack (msgpack) is selected for internal record serialization. It provides compact binary encoding (significantly smaller than JSON), schema-less flexibility (records can evolve without migration), and fast encode/decode performance. The rmp-serde crate provides seamless integration with Rust's serde ecosystem.

Decision: External Archive Format (CMA)

The Cerememory Archive (.cma) format is a self-contained SQLite database file with the following structure:

```
-- CMA file structure (SQLite)

CREATE TABLE cma_meta (
    key TEXT PRIMARY KEY,
    value TEXT
);
-- Contains: version, created_at, cerememory_version, record_count, checksum

CREATE TABLE records (
    id TEXT PRIMARY KEY,
    store TEXT NOT NULL,
    data BLOB NOT NULL,        -- MessagePack-encoded MemoryRecord
    created_at TEXT NOT NULL,
    fidelity REAL NOT NULL
);

CREATE TABLE associations (
    source_id TEXT NOT NULL,
    target_id TEXT NOT NULL,
```

```
    type TEXT NOT NULL,
    weight REAL NOT NULL,
    PRIMARY KEY (source_id, target_id, type)
);

CREATE TABLE evolution_state (
    key TEXT PRIMARY KEY,
    data BLOB NOT NULL          -- MessagePack-encoded evolution parameters
);
```

SQLite is chosen as the archive container because it is the most widely deployed database engine in the world, it is a single file, it is readable from every programming language, and it provides built-in integrity checking. A .cma file can be inspected with any SQLite client for debugging or auditing.

When encryption is enabled, the .cma file is encrypted using SQLCipher (AES-256-CBC) with a user-provided key.

Consequences

- **Positive:** MessagePack is 30-50% smaller than JSON and an order of magnitude faster to parse. Critical for a system storing potentially millions of records.
- **Positive:** SQLite-based archive is universally readable and inspectable. Users can verify exactly what is in their archive.
- **Positive:** the archive format is independent of the internal storage engine. Even if the core engine changes, archives remain compatible.
- **Negative:** MessagePack is not human-readable. Mitigated by the introspection API which provides human-readable views of stored data.

ADR-008: Repository and Crate Structure

Status	**Accepted**
Date	2026-03-12
Decision	**Cargo workspace monorepo with modular crate separation**

Context

Cerememory has multiple distinct components (core engine, individual stores, decay engine, association engine, transport bindings, LLM adapters, SDKs) that should be independently compilable and testable, but closely coordinated.

Decision

A Cargo workspace monorepo with the following crate structure:

```
cerememory/
  Cargo.toml                    # Workspace root
  docs/
    whitepaper.pdf
    cmp-spec-v1.pdf
    adr/
  crates/
    cerememory-core/             # Core types, traits, CMP protocol types
    cerememory-store-episodic/   # Episodic store implementation
    cerememory-store-semantic/   # Semantic (graph) store implementation
    cerememory-store-procedural/ # Procedural store implementation
    cerememory-store-emotional/  # Emotional metadata layer
    cerememory-store-working/    # Working memory cache
    cerememory-decay/            # Decay engine
    cerememory-association/      # Spreading activation engine
    cerememory-evolution/        # Evolution engine
    cerememory-index/            # Hippocampal coordinator (cross-store index)
    cerememory-engine/           # Orchestrator: assembles all stores + engines
    cerememory-transport-http/   # HTTP/REST binding (axum)
    cerememory-transport-grpc/   # gRPC binding (tonic)
    cerememory-archive/          # CMA export/import
    cerememory-cli/              # CLI tool
  bindings/
    python/                      # PyO3 SDK
    typescript/                  # napi-rs SDK
  adapters/
    adapter-claude/             # Anthropic Claude adapter
```

```
  adapter-openai/             # OpenAI GPT adapter
  adapter-gemini/             # Google Gemini adapter
tests/
  integration/                 # Cross-crate integration tests
benches/                       # Performance benchmarks
```

Consequences

- **Positive:** each crate has a clear responsibility. Contributors can work on a single store or engine without understanding the entire codebase.
- **Positive:** crates can be compiled and tested independently. CI builds are parallelizable.
- **Positive:** users can depend on only the crates they need (e.g., cerememory-core for protocol types only).
- **Negative:** many crates to manage. Mitigated by workspace-level dependency management and CI automation.

Summary of Decisions

ADR	Decision	Key Driver
001	Rust as core language	Memory safety + zero-cost concurrency + embeddability
002	Embedded multi-engine storage	Local-first, zero dependencies, optimal per-store
003	Tokio + rayon dual-pool concurrency	I/O and compute isolation
004	Weighted BFS spreading activation	Bounded latency, brain-inspired propagation
005	Modified power-law decay model	Empirical fit to human forgetting data
006	Trait-based LLM adapters + Python/TS SDKs	LLM-agnostic, community accessibility
007	MessagePack internal + SQLite archive	Compact, fast, universally inspectable
008	Cargo workspace monorepo	Modular development, independent testing

---

These decisions are living documents. They will be revisited as the project evolves.

github.com/co-r-e/cerememory